

**PROJECT REPORT ON**

# **TEMPERATURE & HUMIDITY MONITORING SYSTEM WITH VOICE ALERT**

---

*Submitted in partial fulfilment of the requirements of the degree of*  
**MASTER OF COMPUTER APPLICATIONS**

**SUBMITTED BY**

| NAME                  | REG. NUMBER    |
|-----------------------|----------------|
| Prabin Kumar Nayak    | 1ETMCA20251013 |
| Bijayandra Patra      | 1ETMCA20251007 |
| Mukesh Jena           | 1ETMCA20251017 |
| Satya Ranjan Satpathi | 1ETMCA20251008 |

**UNDER THE GUIDANCE OF**  
**MR. BISWAJEEBAN MISHRA**  
Faculty of Engineering and Technologies

---

**SRI SRI UNIVERSITY**  
CUTTACK, ODISHA – 754006

# CONTENTS

1. Project Overview
2. Scope Definition
3. Requirements Summary
4. System Architecture
5. Development Plan
6. Testing and Quality Strategy
7. Deployment and Operations
8. Risk Management
9. Change Management
10. Conclusion

## Abstract

The Internet of Things (IoT) has revolutionized the way we interact with our environment. This project, Temperature and Humidity Monitoring System with Voice Alert, leverages IoT principles to build a real-time environmental monitoring solution using a DHT11 sensor and Arduino UNO microcontroller.

The system continuously reads temperature and humidity values from the DHT11 sensor connected to the Arduino UNO board. The sensor data is transmitted to a connected PC via serial communication. A Python script running on the PC reads this data, evaluates threshold conditions, and triggers voice alerts using the pyttsx3 text-to-speech engine.

When the temperature exceeds 37°C, the system announces a warning alert. When it drops below 20°C, a cold temperature alert is triggered. For normal conditions, a standard informational message is spoken. The system avoids repeated alerts for the same state to prevent unnecessary noise.

This project demonstrates the practical integration of hardware sensors, serial communication, and software-based voice processing to create an intelligent, low-cost environmental monitoring solution.

# 1. PROJECT OVERVIEW

## Business Problem and Motivation

In everyday environments, temperature fluctuations can pose serious health risks. Extreme heat or cold can impact human comfort, safety, and productivity. Traditional manual monitoring systems require constant human attention and are often unreliable. There is a clear need for an automated, real-time system that can alert users when conditions become dangerous.

This project addresses the problem by building a compact, affordable IoT-based monitoring solution. Using the DHT11 sensor and Arduino UNO, the system collects environmental data and uses text-to-speech technology to provide instant audio feedback to users, removing the need for them to check readings manually.

## Objectives

1. To design and implement a temperature and humidity monitoring system using IoT components.
2. To read real-time sensor data from DHT11 via Arduino UNO.
3. To process serial data using a Python script running on a PC.
4. To provide automated voice alerts based on temperature thresholds.
5. To build a low-cost, portable prototype demonstrating embedded IoT concepts.

## Success Criteria

1. Accurate and continuous reading of temperature and humidity values.
2. Successful transmission of sensor data via serial communication.
3. Correct voice alert generation based on defined thresholds.
4. Stable and uninterrupted system operation.
5. No repeated voice alerts for the same temperature state.

## Stakeholders

Students and Researchers: Primary users experimenting with IoT sensor systems.

Home Users: Individuals monitoring household temperature comfort.

Healthcare Workers: Environments where temperature control is critical.

Faculty / Evaluators: Academic stakeholders assessing the project.

## 2. SCOPE DEFINITION

### A. In-Scope

1. Real-time reading of temperature and humidity from DHT11 sensor.
2. Serial communication between Arduino UNO and PC via USB.
3. Python-based serial data parsing and threshold evaluation.
4. Voice alert generation using pyttsx3 text-to-speech engine.
5. State-based alerting to prevent redundant voice messages.

### B. Out-of-Scope

1. Cloud data storage or remote monitoring via the internet.
2. Mobile application notifications or dashboards.
3. Integration with HVAC or air conditioning control systems.
4. AI-based predictive environmental analysis.
5. Display modules (LCD/OLED) for visual output.

### C. Assumptions and Constraints

#### Assumptions:

1. A stable USB power supply is available at all times.
2. The Arduino board is connected to COM6 at 9600 baud rate.
3. The DHT11 sensor provides accurate readings under normal conditions.

#### Constraints:

1. Limited to the detection range and accuracy of the DHT11 sensor ( $\pm 2^{\circ}\text{C}$ ,  $\pm 5\%$  RH).
2. Voice alerts require PC speakers to be functional.
3. Prototype-level implementation without industrial-grade hardware.

### 3. REQUIREMENTS SUMMARY

#### A. Functional Requirements

1. The DHT11 sensor shall continuously measure temperature and humidity.
2. The Arduino UNO shall read sensor data and transmit it over serial port.
3. The Python script shall read serial data and decode temperature values.
4. The system shall compare temperature against thresholds:  $\geq 37^{\circ}\text{C}$  (hot),  $\leq 20^{\circ}\text{C}$  (cold).
5. Voice alerts shall be triggered only on state change to avoid repetition.
6. The system shall handle exceptions gracefully and continue operation.

#### B. Non-Functional Requirements

Reliability: The system must operate continuously without crashes.

Performance: Alert latency from sensor read to voice output should be under 3 seconds.

Scalability: Future sensors (e.g., CO2, motion) can be added to the pipeline.

Maintainability: Codebase is modular and easy to update.

Cost Efficiency: Total component cost is under ₹500.

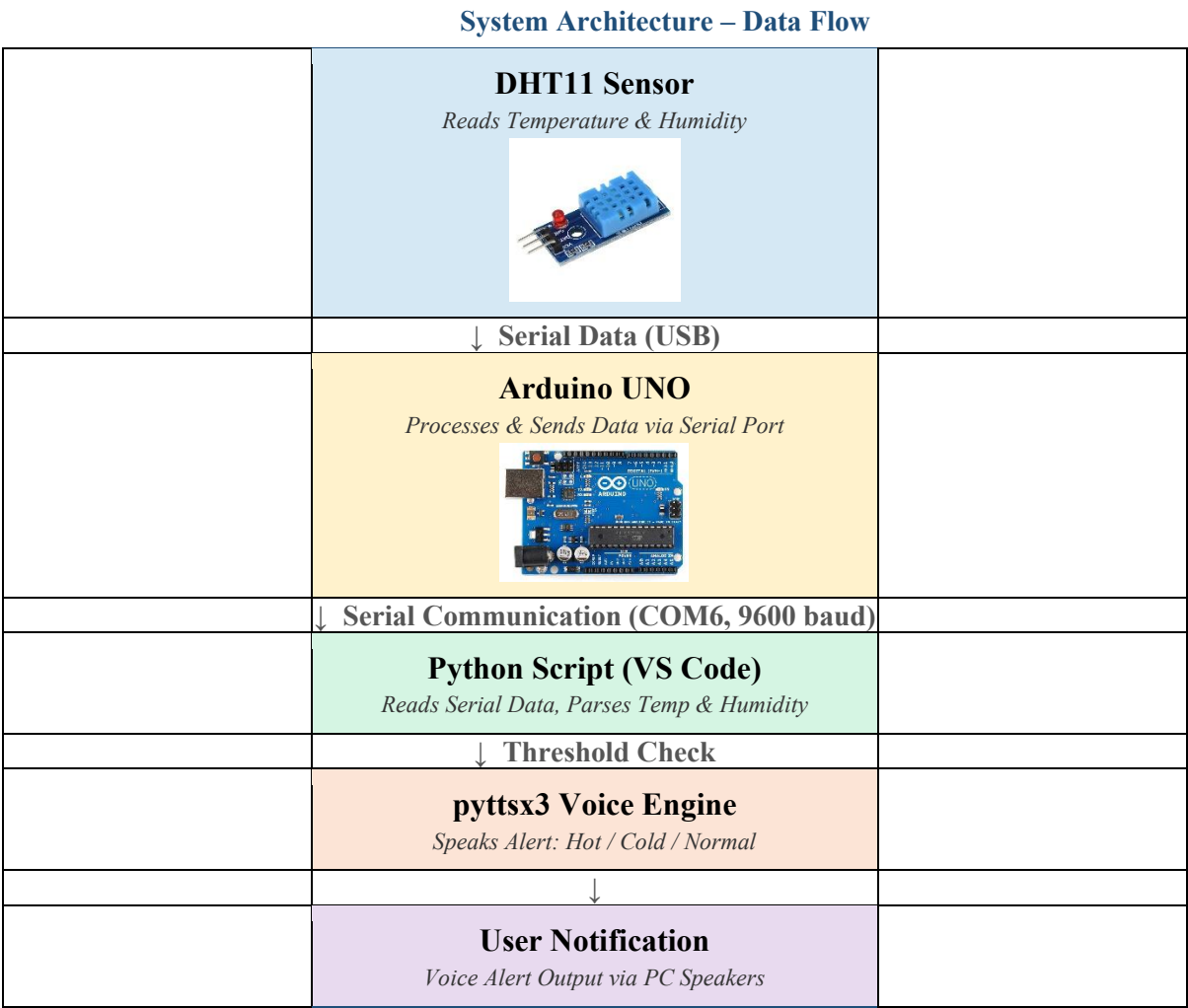
#### C. Hardware Components Used

| Component         | Quantity | Purpose                        |
|-------------------|----------|--------------------------------|
| Arduino UNO       | 1        | Microcontroller board          |
| DHT11 Sensor      | 1        | Temperature & Humidity sensor  |
| Jumper Wires      | 3        | Circuit connections            |
| Arduino USB Cable | 1        | Power & serial communication   |
| PC / Laptop       | 1        | Runs Python voice alert script |

## 4. SYSTEM ARCHITECTURE

The system architecture follows a layered IoT design pattern with four core stages: Sensing, Processing, Communication, and Output. Each layer has a specific role and communicates with adjacent layers.

### 4.1 Architecture Diagram



### 4.2 Layer Description

#### Sensing Layer (DHT11 Sensor):

The DHT11 digital sensor reads ambient temperature (0-50°C range) and relative humidity (20-90% RH). It communicates digitally with the Arduino using a single data wire. A 10kΩ pull-up resistor stabilizes the data line.

#### Processing Layer (Arduino UNO):

The Arduino UNO microcontroller runs a sketch that initializes the DHT library, reads sensor values every 2 seconds, and transmits them as formatted strings (e.g., Humidity:82.00 ,Temp:34.20) over the serial port at 9600 baud.

#### Communication Layer (USB Serial):

The Arduino connects to the PC via USB cable. The serial communication protocol operates at 9600 baud rate on COM6. Python reads this using the pyserial library, decoding and stripping the incoming byte stream.

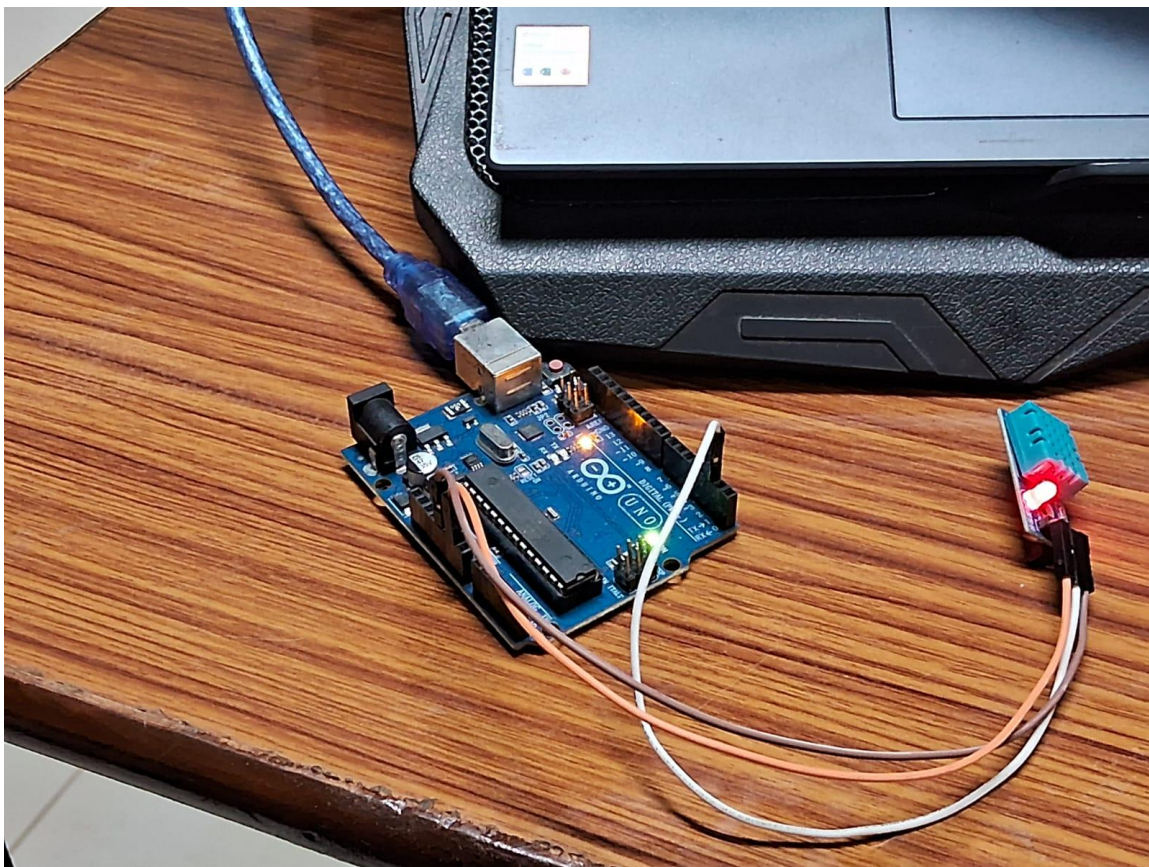
### **Output / Alert Layer (Python + pyttsx3):**

The Python script parses incoming serial data, extracts the temperature value, compares it against thresholds, and uses pyttsx3 to generate platform-native text-to-speech voice alerts. State tracking ensures alerts are not repeated.

## **4.3 Working Procedure**

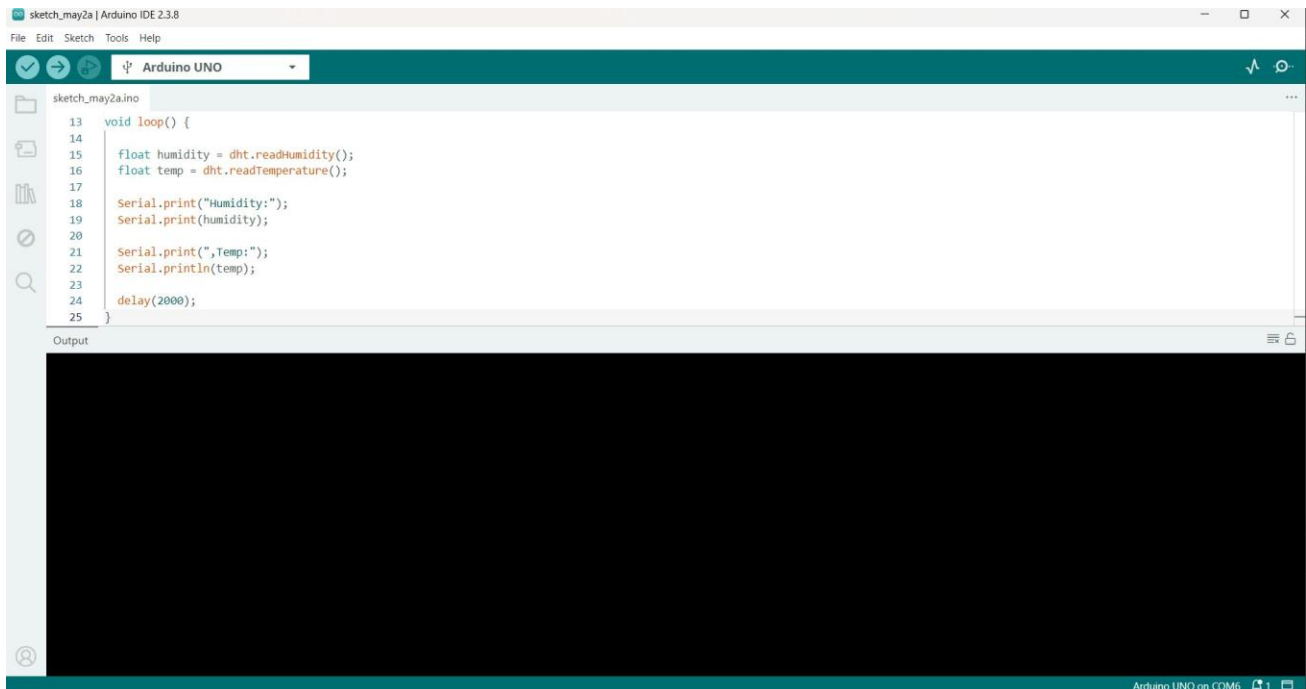
1. DHT11 sensor continuously measures temperature and humidity.
2. Arduino reads sensor data every 2 seconds and formats it as a serial string.
3. Data is transmitted over USB serial to the connected PC (COM6, 9600 baud).
4. Python script reads and decodes the serial stream, extracting temperature.
5. Threshold evaluation:  $\geq 37^{\circ}\text{C}$   $\rightarrow$  hot alert,  $\leq 20^{\circ}\text{C}$   $\rightarrow$  cold alert, else  $\rightarrow$  normal.
6. If the temperature state has changed, pyttsx3 speaks the corresponding alert message.
7. System continues running in a loop, sleeping 1 second between iterations.

## **4.4 Hardware Setup**



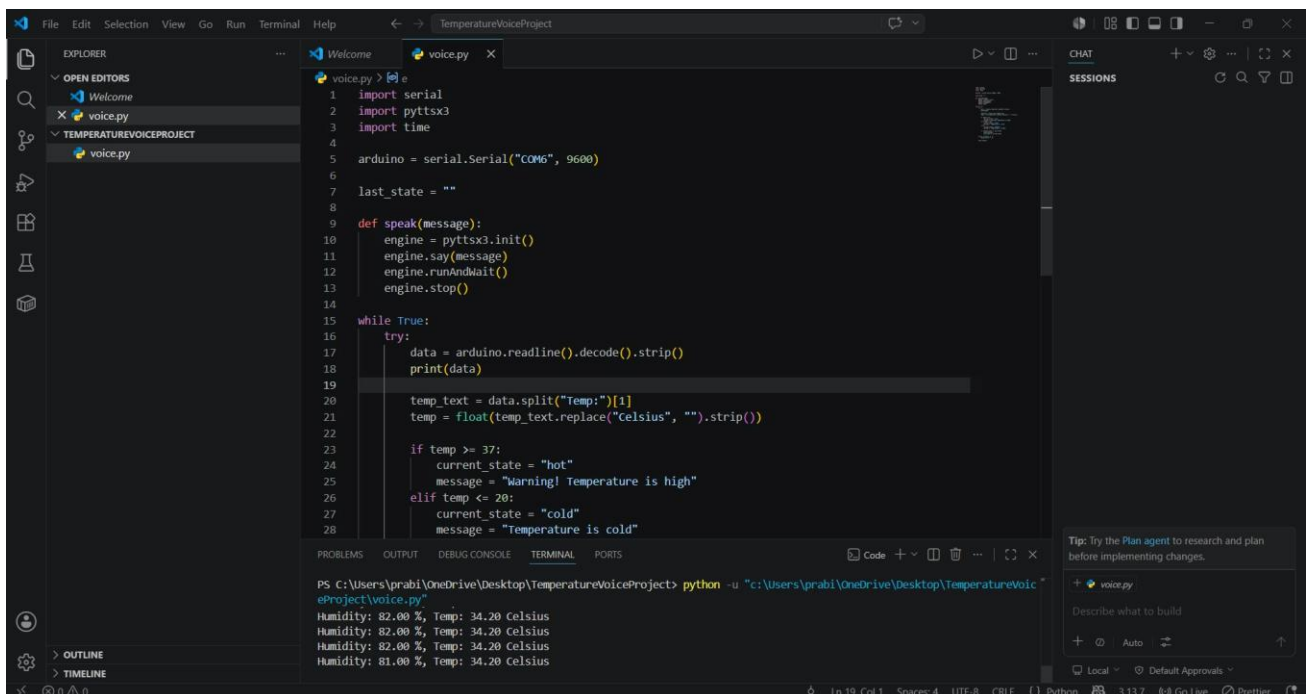
## 4.5 Arduino Code (sketch\_may2a.ino)

The following screenshot shows the Arduino sketch running in Arduino IDE 2.3.8, reading DHT11 sensor data and transmitting it over serial:



## 4.6 Python Voice Alert Script (voice.py)

The Python script runs in VS Code and handles serial data reading, threshold evaluation, and voice alert generation:



## 5. DEVELOPMENT PLAN

### Development Methodology

The project follows the Agile development methodology, enabling iterative development, rapid prototyping, and continuous testing at each sprint.

### Sprint 1: Requirement Analysis

1. Understanding project objectives and sensor requirements.
2. Identifying hardware components: DHT11, Arduino UNO, USB cable, jumper wires.
3. Researching DHT library and serial communication protocols.

### Sprint 2: Hardware Setup

4. Connecting DHT11 sensor to Arduino UNO using jumper wires.
5. Installing and configuring Arduino IDE 2.3.8.
6. Verifying hardware connections and power supply.

### Sprint 3: Software Development

7. Writing Arduino sketch to read DHT11 sensor data.
8. Implementing serial print for humidity and temperature values.
9. Developing Python script for serial data reading and parsing.
10. Integrating pyttsx3 for voice alert generation.
11. Implementing state-based alert logic.

### Sprint 4: Testing and Debugging

12. Testing sensor accuracy and data transmission.
13. Verifying threshold conditions and voice alerts.
14. Debugging serial communication errors and exception handling.
15. Optimizing delay and polling intervals.

### Sprint 5: Deployment and Documentation

16. Final hardware and software integration.
17. Report preparation and documentation.
18. Project demonstration and review.

## 6. TESTING AND QUALITY STRATEGY

### Test Cases

| Test Case                                       | Expected Result                           | Status |
|-------------------------------------------------|-------------------------------------------|--------|
| Temperature $\geq 37^{\circ}\text{C}$           | Voice alert: Warning! Temperature is high | Pass   |
| Temperature $\leq 20^{\circ}\text{C}$           | Voice alert: Temperature is cold          | Pass   |
| Normal range ( $21\text{-}36^{\circ}\text{C}$ ) | Voice alert: Temperature is normal        | Pass   |
| Serial data reading                             | Data parsed and printed correctly         | Pass   |
| Duplicate state                                 | Alert not repeated for same state         | Pass   |
| Hardware connection                             | Arduino detected on COM6 at 9600 baud     | Pass   |

### Acceptance Criteria

1. DHT11 sensor accurately reads temperature and humidity values.
2. Python correctly parses the serial string and extracts temperature.
3. Voice alerts are generated promptly upon threshold crossing.
4. No duplicate voice messages for the same temperature state.
5. System handles errors gracefully without crashing.

## 7. DEPLOYMENT AND OPERATIONS

The prototype is deployed using Arduino UNO hardware connected to a DHT11 temperature and humidity sensor via three jumper wires. The Arduino is powered and communicates with the PC through an Arduino USB cable. The Python script runs on the connected laptop using VS Code.

The system runs continuously in a loop, monitoring temperature changes and delivering voice alerts in real time. As seen in the terminal output (Figure 4), the system successfully read values such as Humidity: 82.00 %, Temp: 34.20 Celsius in continuous cycles.

### Future Enhancements

6. Display temperature on an LCD or OLED screen for visual monitoring.
7. Send data to a cloud platform (e.g., ThingSpeak, Firebase) for logging.
8. Build a mobile application for remote monitoring and push notifications.
9. Add alarm or buzzer hardware for audible hardware-level alerts.
10. Integrate humidity-based alerts alongside temperature.
11. Add a web dashboard for historical data visualization.

## 8. RISK MANAGEMENT

| Risk                 | Impact                                | Mitigation                                     |
|----------------------|---------------------------------------|------------------------------------------------|
| Sensor misreading    | Incorrect temperature/humidity values | Regular calibration of DHT11 sensor            |
| Serial port conflict | Python fails to connect to Arduino    | Ensure COM port is free before running         |
| Power interruption   | System shutdown mid-operation         | Use stable USB power supply                    |
| Python crash         | Voice alert stops                     | Wrap in try/except with graceful error logging |

## 9. CHANGE MANAGEMENT

Any modifications to the hardware configuration, sensor wiring, Arduino sketch, or Python script should be documented before implementation. Version control practices (e.g., Git) should be adopted to track code changes and maintain project stability.

The team should maintain a change log recording the date, author, nature of change, and test results. Before deploying changes to the running system, all modified components must be tested in isolation first.

## 10. CONCLUSION

The Temperature and Humidity Monitoring System with Voice Alert successfully demonstrates a practical, low-cost IoT application integrating embedded hardware with software-based processing. The DHT11 sensor attached to an Arduino UNO provides continuous real-time environmental data, which is transmitted via USB serial communication to a Python script on a PC.

The Python script intelligently evaluates temperature thresholds and uses the pyttsx3 text-to-speech library to deliver context-sensitive voice alerts: a warning when temperature is high ( $\geq 37^{\circ}\text{C}$ ), a cold alert when temperature is low ( $\leq 20^{\circ}\text{C}$ ), and a normal status message for comfortable conditions. The state-tracking mechanism prevents redundant alerts, making the system user-friendly and non-intrusive.

The project was developed by a team of four MCA students and demonstrates hands-on integration of embedded systems programming (Arduino C++), Python scripting, serial communication, and text-to-speech synthesis. The total hardware cost remains under ₹500, making it accessible for educational purposes.

Future iterations of this system could include cloud integration, mobile notifications, display modules, and multi-sensor support to build a more comprehensive environmental monitoring platform. Overall, this project fulfils its objectives and highlights the immense potential of IoT technologies in everyday monitoring and safety applications.

---

— End of Report —